Robot autobalanceante con Aprendizaje por Refuerzo Profundo

Avance del 23 de octubre · Mario Betancourt

> **MODIFICAR**

Índice

1. Propuesta
2. Entorno de simulación (MuJoCo)
3. Entorno de Gymnasium
4. DDPG
5. PPO
6. Entrenamiento

1. Propuesta

En este proyecto se diseñan e implementan algoritmos de aprendizaje por refuerzo profundo para controlar el movimiento de un robot autobalanceante. El robot consiste en un cuerpo rígido con dos ruedas acopladas, impulsadas por dos motores con cajas reductoras. Para medir su orientación, el robot utiliza una **IMU** (Unidad de Medición Inercial), que proporciona en tiempo real el vector de aceleración lineal, el vector de velocidad angular y el vector de campo magnético en la posición donde se encuentra. El algoritmo, programado en un microcontrolador, usa esta información para controlar los motores y recibir un refuerzo si la acción contribuye a que el robot se mantenga balanceado sobre su eje vertical.

1.1. Algoritmos

Se propone comparar el rendimiento de los cuatro algoritmos vistos durante la sesión de Deep RL parte 2: **DDPG, TD3, SAC y PPO**. El resultado final de esta comparación debe ser una serie de gráficas que muestren la recompensa en función del número de episodios de entrenamiento. A través de estas gráficas será posible determinar qué algoritmo *aprende más rápido* a cumplir con la tarea y, en estado estacionario, cuál termina la misión con mejor rendimiento.

1.2. Fuentes de incertidumbre

Una de las principales motivaciones para usar algoritmos de aprendizaje por refuerzo en este problema es la cantidad importante de parámetros con incertidumbre asociada a diferencias de implementación y a factores del ambiente. Se desea que, si los algoritmos se implementan en un robot físico, este pueda balancearse sin importar en qué parte del chasis se coloque la IMU.

1.3. Antecedentes

Este proyecto está inspirado en la serie de videos de Digikey y Shawn Hymel *"Reinforcement Learning for Robotics"*, donde se implementa un robot autobalanceante con el kit **M5Stack BALA2** de Digikey usando un algoritmo PPO. En este proyecto se utilizan los archivos STL de la serie, pero la implementación de los cuatro algoritmos es propia y documentada por los integrantes del equipo.

2. Entorno de simulación

2.1. Motor de físicas MuJoCo

MuJoCo (*Multi-Joint dynamics with Contact*) es un motor de física de código abierto desarrollado por Google DeepMind. Está diseñado para realizar simulaciones rápidas y precisas. Sus fortalezas principales son su simulación realista del contacto físico, el uso de formatos de modelado simples y compatibles con otros motores de física, la inclusión de herramientas de visualización y, posiblemente lo más importante, su **fácil integración con IA**. MuJoCo puede usarse como librería de Python e integrarse directamente con Gymnasium.

Para utilizar MuJoCo en un programa de Python, es necesario instalarlo en la terminal de nuestra computadora:

```
pip install mujoco gymnasium
```

In [2]:
```python
# Esta celda descarga Mujoco y Gymnasium en su computadora si todavía no los tiene
# Si ya tiene instaladas ambas librerías, aparecerá el mensaje "MuJoCo y Gymnasium ya están insta
# [Celda generada con Copilot]

import subprocess
import sys

import importlib.util

paquetes = {
    "mujoco": "mujoco",
    "gymnasium": "gymnasium",
}

faltantes = [
    paquete
    for modulo, paquete in paquetes.items()
    if importlib.util.find_spec(modulo) is None
]

if faltantes:
    subprocess.check_call([sys.executable, "-m", "pip", "install", *faltantes])
    print(f"Instalados: {', '.join(faltantes)}")
else:
    print("MuJoCo y Gymnasium ya están instalados.")
```

```
MuJoCo y Gymnasium ya están instalados.
```

2.2. Modelado 3D en CAD

MuJoCo trabaja con mallas que describen los objetos que componen el modelo tridimensional del robot. Estos modelos se pueden generar en software de diseño asistido por computadora (CAD, por sus siglas en inglés). La mayoría de programas CAD de uso común en ingeniería (p. ej. SolidWorks, AutoCAD, Autodesk Inventor, Autodesk Fusion, CATIA, entre otros) pueden exportar archivos **STL** (*Standard Tessellation Language*). El formato STL define la geometría de la superficie de un objeto 3D mediante una malla de triángulos interconectados... ¡Es justo lo que MuJoCo necesita! Por practicidad, se recomienda generar un archivo STL por cada parte con movimiento independiente en el robot. Esto se explicará con más detalle en la siguiente sección.

Una observación importante es que los modelos de malla se vuelven progresivamente más complicados y, por lo tanto, difíciles de usar en motores de física conforme aumenta la cantidad de detalles pequeños. En las regiones con detalles finos hay más triángulos, lo que requiere más cómputo para resolver los contactos. Esto impacta, por ejemplo, en el modelado de las bandas de rodadura de las cubiertas de las ruedas. En su lugar, podemos considerar estas "deformidades" aleatorizando el dominio, como se explicará más adelante.

Respecto a la serie de videos de Shawn Hymel, en esta parte del proyecto omitimos el proceso de diseñar en 3D el cuerpo del robot y las ruedas. Utilizamos directamente los archivos STL de la serie para importarlos al entorno.

Además de los archivos STL, necesitamos información sobre las propiedades físicas de cada parte del robot. Por cada parte móvil del modelo simplificado se requiere su **masa** m, las coordenadas de su **centro de masa** $\mathbf{r}_{cm}$ y su **tensor de inercia** I (o, en su defecto, sus momentos principales I_{xx}, I_{yy}, I_{zz}). Estas tres cantidades normalmente se pueden obtener en software CAD. En el proyecto original se obtuvieron con la API de FreeCAD en Python, aunque conociendo otras opciones, este procedimiento podría incluso ser innecesario.

2.3. Formato de modelo MuJoCo

Para definir las características del entorno, MuJoCo utiliza el formato nativo **MJCF**, un lenguaje basado en XML. En este archivo el usuario puede definir:

- **Uniones entre elementos**
- **Sensores y actuadores**
- **Parámetros físicos de la simulación**: gravedad, diferencial de tiempo
- **Visualización y renderizado**

MuJoCo ofrece amplia documentación sobre cómo configurar cada uno de estos parámetros. A pesar de ello, aquí se presenta una breve introducción.

> **Uso de IA:** El esqueleto del archivo MJCF se generó con Gemini, consultándole por una serie de pasos para construir el formato. Los pasos se resumen a continuación (escritos a mano).

Observe en el archivo `autobalance.xml` que el esqueleto del MJCF define primero los parámetros del compilador (ángulos en radianes, la carpeta donde se encuentran las mallas y el diferencial de tiempo

máximo). Luego se configura la opción de diferencial de tiempo.

La importación de archivos STL a MuJoCo se realiza dentro del bloque `asset`. Cada malla se importa mediante una línea `<mesh ...>`, donde se indica el nombre del archivo y su escala. Observe que se aplica un factor de escalamiento de 0.001 en los tres ejes, ya que el milímetro es la unidad habitual en softwares de diseño. Al multiplicar por 0.001 (es decir, dividir entre 1000) se obtienen dimensiones en metros.

En `worldbody` se definen tanto las propiedades del entorno como el robot. Observe que en `geom` se define la geometría del piso, incluyendo su tamaño y su coeficiente de fricción.

El cuerpo del robot también se define dentro de `worldbody`. Las **masas** se representan mediante `<inertial>` y las geometrías para colisiones mediante `geom`. Cualquier ubicación relevante que no corresponda a una malla ni a un objeto con masa puede definirse mediante `site`. En el robot definido hay un único `site`, que corresponde a la ubicación física del sensor IMU. Las ruedas se acoplan al chasis mediante **uniones** (`joint`). Al igual que en los eslabones de un mecanismo, el tipo de unión dicta cuántos grados de libertad tiene y qué tipo de movimiento puede realizar (traslacional, rotacional o complejo). Una unión de tipo `hinge` (bisagra) solo puede rotar alrededor de un eje. Esta es, precisamente, la que se utiliza en las ruedas del robot.

Finalmente, el bloque `actuator` define los motores de cada rueda. Entre los parámetros de `motor` se incluye la **razón de la caja reductora**, que representa la reducción de velocidad del eje del motor al eje de salida. También se define el rango de control del motor.

En el bloque `sensor` se agregan el acelerómetro y el giroscopio en el `site` definido anteriormente. En este bloque también se agregan la posición y la velocidad de los motores mediante encoders. Sin embargo, como se indica en los videos, estos últimos cuatro valores solo existen dentro de la simulación.

2.4. Aspectos de implementación que pueden cambiar

Dado que este proyecto está pensado para implementar un agente con cuerpo (*embodied agent*), es importante notar cómo el robot puede medir el estado de movimiento de sus motores.

Para medir la posición y la velocidad angular de un motor eléctrico, el método utilizado en el kit de Digikey es emplear **encoders**: sensores optomecánicos con ruedas ranuradas que codifican la posición angular de un eje mediante estados de luz y oscuridad.

No obstante, si queremos ser más tacaños, vale la pena hacer un pequeño cambio en esta parte. En un robot más simple, el control de los motores podría realizarse mediante una técnica conocida como **PWM** (*Pulse Width Modulation*). En ella, el voltaje entregado a los motores se produce de forma pulsada, con un ancho de pulso que varía entre 0 y el periodo completo. Esto permite controlar la velocidad del motor, aunque de todas formas se necesita un encoder para determinar el estado de los motores... Por lo tanto, a diferencia del proyecto original, en una implementación física con control de velocidad PWM, el estado del movimiento de los ejes de los motores deja de ser observable y no puede ser incluida en el estado.

3. Entorno de Gymnasium

El archivo `balance_bot_env.py` define la clase `BalanceBotEnv`, que hereda de `gym.Env`.

Espacio de acciones

La acción es un vector bidimensional de tipo `float32`. Cada componente representa el torque normalizado de los motores izquierdo y derecho del robot. Un número positivo indica que el motor gira en una dirección que impulsa el robot hacia adelante, mientras que un número negativo indica que lo impulsa hacia atrás.

$$\mathcal{A} = \{(l, r) \in \mathbb{R}^2 : |l| \leq 1,\ |r| \leq 1\}$$

Espacio de observaciones

Se decidió que en esta implementación no se utilicen encoders de motores, por el costo adicional que imponen en una realización física. Por lo tanto, el sensor principal es la IMU, que entrega el vector de aceleración y el vector de velocidad angular. Recuerde de sus clases de física que la aceleración traslacional puede integrarse una vez para obtener la velocidad traslacional, y una segunda vez para obtener la posición traslacional. De forma similar, la velocidad angular de la IMU puede integrarse una vez para obtener la posición angular. Hay que tener en cuenta que la integración acumula error (*deriva*) con el tiempo.

El espacio de observaciones se diseñó como un vector con el ángulo respecto a la vertical (*pitch*, θ), la velocidad angular respecto a pitch (*pitch rate*, $\dot{\theta}$), la velocidad traslacional del robot y su posición. Estas variables se escogieron porque pueden ayudar a definir una función de recompensa que logre dos objetivos importantes: mantenerse de pie durante un tiempo indefinidamente prolongado y responder a comandos que controlen su velocidad.

> **Pendiente (inconsistencia):** el método `_get_observation` implementado devuelve $[\theta, \dot{\theta}, \omega_l, \omega_r]$, es decir, las velocidades angulares de las ruedas, en lugar de la velocidad y la posición traslacionales. Además, esas velocidades de las ruedas se leen directamente de la simulación. Hay que decidir si se mantiene el diseño descrito arriba (con estimación de posición a partir de la IMU) o si se actualiza el código y esta descripción.

Terminación del episodio

En la inicialización del entorno se define un ángulo máximo de inclinación de 30° (`max_tilt`). Si el robot se inclina a un ángulo mayor durante un episodio de entrenamiento, el episodio termina automáticamente:

$$\text{terminated} = \big(|\theta_t| > \theta_{max}\big), \qquad \theta_{max} = 30^\circ$$

Función de recompensa

La función de recompensa se implementa en `step`. La propuesta de diseño tiene dos componentes:

1. Un castigo proporcional al ángulo de inclinación en cada paso en el que el episodio no ha terminado.
2. Un premio cuando el ángulo de inclinación se mantiene dentro de un margen $\theta_{min} = 2^\circ$ respecto a la vertical.

Con estas dos componentes, la recompensa en el paso t (mientras el episodio no termina) puede escribirse como

$$r_t = -k_1 \, |\theta_t| + k_2 \, 1\,[\,|\theta_t| \leq \theta_{min}\,]$$

donde $k_1, k_2 > 0$ son pesos que todavía deben definirse y $1[\cdot]$ es la función indicadora, que vale 1 si la condición se cumple y 0 si no.

> **Pendiente (inconsistencia):** la función `step` implementada actualmente es un ejemplo básico: devuelve $r_t = 1$ mientras el robot no cae y $r_t = 0$ al caer. La fórmula anterior es la propuesta de diseño y todavía no está implementada. También falta definir la recompensa o penalización al terminar el episodio y los valores de k_1 y k_2.

In [2]:
```python
from balance_bot_env import BalanceBotEnv
from gymnasium.utils.env_checker import check_env


# En esta parte del Script
env = BalanceBotEnv(mjcf_path="autobalance.xml") # Ajusta la ruta a tu archivo XML
check_env(env)
print("¡El entorno de Gymnasium cumple con los estándares correctamente!")
```

```
¡El entorno de Gymnasium cumple con los estándares correctamente!
c:\Users\mario\AppData\Local\Programs\Python\Python310\lib\site-packages\gymnasium\utils\env_checker.py:440: UserWarning: WARN: Not able to test alternative render modes due to the environment not having a spec. Try instantiating the environment through `gymnasium.make`
  logger.warn(
```

Una de las muchas genialidades de MuJoCo es que incluye un simulador con visor integrado, que puede abrirse con `mujoco.viewer`. La siguiente celda define la función que lo muestra. El simulador se mantiene activo hasta que usted lo cierre. Para verlo, descomente la última línea de código.

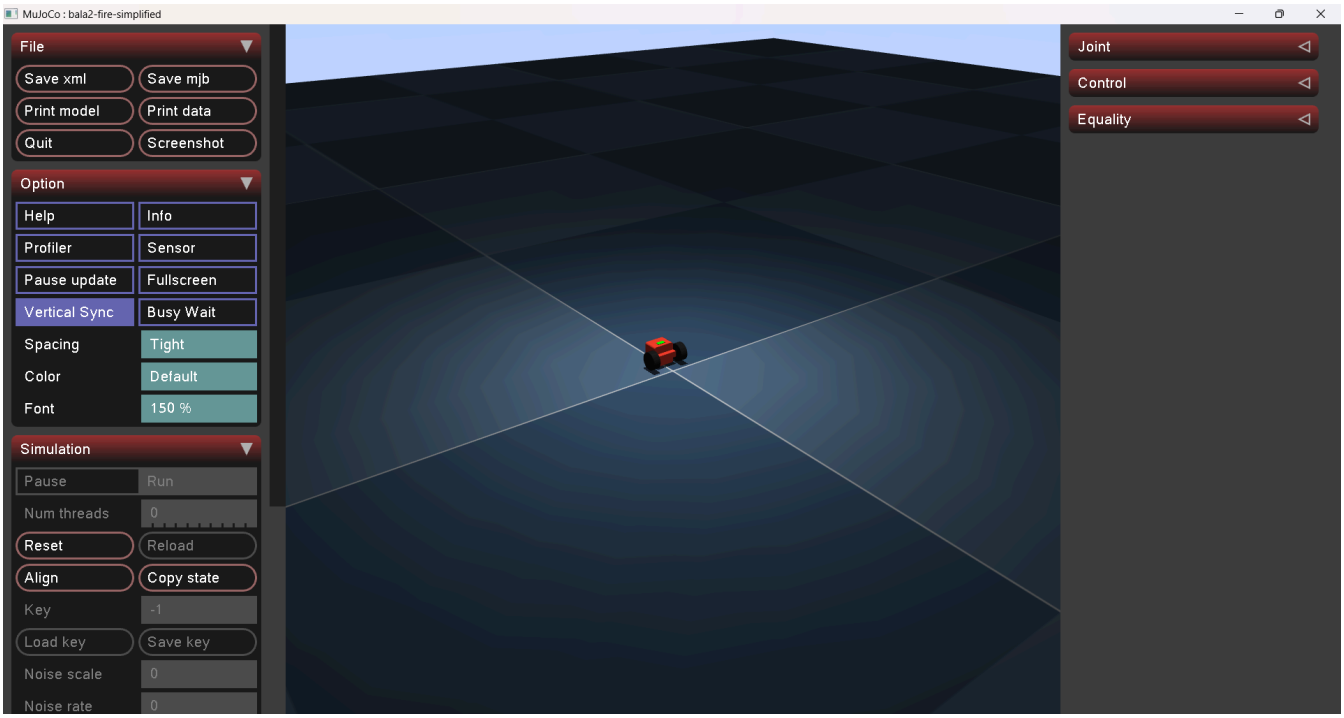
In [ ]:
```python
import mujoco
import mujoco.viewer
import time

def visualizar_simulador_mujoco(mjcf_path="autobalance.xml"):
    """Abre el visor interactivo del robot y ejecuta la simulación."""
    model = mujoco.MjModel.from_xml_path(mjcf_path)
    data = mujoco.MjData(model)

    with mujoco.viewer.launch_passive(model, data) as viewer:
        while viewer.is_running():
            mujoco.mj_step(model, data)
            viewer.sync()
            time.sleep(0.005)


# DESCOMENTAR ESTA LÍNEA PARA VER SIMULADOR
# visualizar_simulador_mujoco()
```

Esto debería mostrar una ventana como la siguiente, en la que se observa al robot parado sobre el origen de coordenadas sin hacer absolutamente nada. Eventualmente el robot podría caerse sobre sí mismo.



4. DDPG

El primer algoritmo a probar en este entorno es **DDPG** (*Deep Deterministic Policy Gradient*). Es una extensión de DQN para dominios con espacios de acciones continuos. A diferencia de DQN, DDPG utiliza dos aproximadores de funciones: uno para la política (el **actor**) y otro para la función de valor de acción (el **crítico**). Por esta razón es un método *actor-crítico*. Además, DDPG utiliza los dos mecanismos que DQN emplea para evitar la inestabilidad de la tríada mortal en la mayoría de los casos: **redes objetivo** (*target networks*) y **repetición de experiencia** (*experience replay*).

En total, DDPG utiliza *cuatro* redes neuronales: una red online y una red objetivo para el actor, y una red online y una red objetivo para el crítico. Las redes online son las que se entrenan.

Abajo se muestra una imagen de la arquitectura de DDPG. El actor online produce la acción determinista $\mu(s \mid \theta^\mu)$, que es evaluada por el crítico online $Q(s, a \mid \theta^Q)$. El crítico proporciona el gradiente del valor de acción con respecto a la acción, y ese gradiente se propaga hacia los parámetros del actor para actualizarlos con un optimizador. Esto corresponde a un proceso de *policy gradient*: la política se parametriza con un vector de pesos θ^μ y se aplica un ascenso por gradiente para encontrar la dirección de máximo incremento del valor esperado. Esta es, de hecho, la solución de DDPG a la necesidad de reemplazar la función $\arg\max$ que usan Q-Learning y DQN. Así, la red del actor aprende a aproximar la política óptima.

El gradiente de la política determinista que se usa para actualizar el actor es

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_s \left[\nabla_a Q(s, a \mid \theta^Q)\big|_{a=\mu(s|\theta^\mu)} \nabla_{\theta^\mu} \mu(s \mid \theta^\mu) \right]$$

Una pregunta natural al reflexionar sobre este algoritmo es: si las redes online del actor y del crítico hacen la mayor parte del trabajo, ¿cuál es el propósito de las redes objetivo? La respuesta es la **estabilidad**. Las redes objetivo cambian lentamente, de modo que durante un número de pasos el entrenamiento de las redes online se parece a un problema de aprendizaje supervisado con objetivos relativamente fijos.

Para el crítico, el objetivo de regresión es

$$y_t = r_t + \gamma\, Q'\big(s_{t+1},\ \mu'(s_{t+1} \mid \theta^{\mu'}) \mid \theta^{Q'}\big), \qquad L(\theta^Q) = \frac{1}{N}\sum_{i=1}^{N}\big(y_i - Q(s_i, a_i \mid \theta^Q)\big)^2$$

donde Q' y μ' son las redes objetivo del crítico y del actor, respectivamente, y γ es el factor de descuento.

Las redes objetivo se actualizan de forma suave (*soft update*, o promedio de Polyak) después de cada paso de entrenamiento:

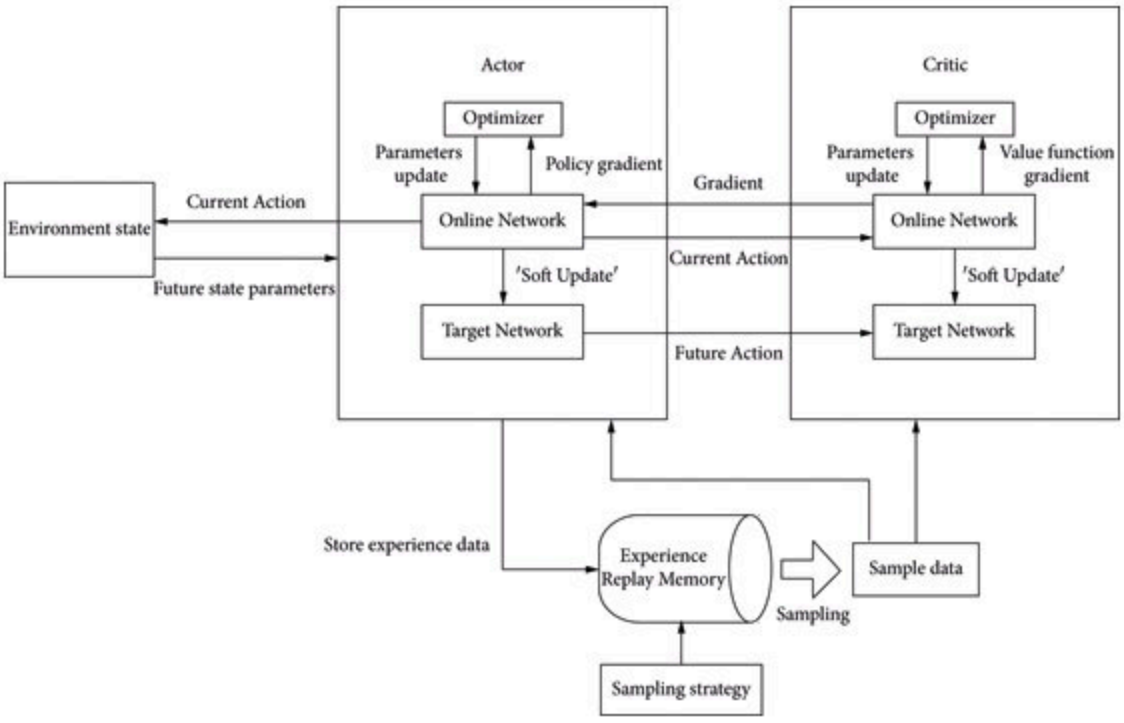
$$\theta' \leftarrow \tau\,\theta + (1-\tau)\,\theta', \qquad \tau \ll 1$$

En el código, este factor corresponde a `tau = 0.005`.

El nombre DDPG resulta de que el algoritmo es básicamente una combinación de *Deep Q-Network* (DQN) y *Deterministic Policy Gradient* (DPG).

Interesantemente, en las implementaciones de PyTorch, dada la red neuronal que aproxima $Q(s, a)$, no es necesario calcular las derivadas parciales de forma iterativa. En su lugar, se utiliza *backpropagation* para obtener el gradiente de manera eficiente.

En esta implementación DDPG se realiza en un script separado `ddpg.py`.



Algorithm 1 Deep Deterministic Policy Gradient

Input: initial policy parameters θ, Q-function parameters ϕ, empty replay buffer $\mathcal{D}$
Set target parameters equal to main parameters $\theta_{\text{targ}} \leftarrow \theta$, $\phi_{\text{targ}} \leftarrow \phi$
repeat
  Observe state s and select action $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{Low}, a_{High})$, where $\epsilon \sim \mathcal{N}$
  Execute a in the environment
  Observe next state s', reward r, and done signal d to indicate whether s' is terminal
  Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
  If s' is terminal, reset environment state.
  if it's time to update **then**
    for however many updates **do**
      Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
      Compute targets

$$y(r, s', d) = r + \gamma(1 - d)Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s'))$$

      Update Q-function by one step of gradient descent using

$$\nabla_\phi \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \left(Q_\phi(s, a) - y(r, s', d)\right)^2$$

      Update policy by one step of gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} Q_\phi(s, \mu_\theta(s))$$

      Update target networks with

$$\phi_{\text{targ}} \leftarrow \rho\phi_{\text{targ}} + (1 - \rho)\phi$$
$$\theta_{\text{targ}} \leftarrow \rho\theta_{\text{targ}} + (1 - \rho)\theta$$

    end for
  end if
until convergence

Referencia: Spinning Up – DDPG

5. PPO

El segundo algoritmo a implementar en este cuadernillo es **PPO** (*Proximal Policy Optimization*). Es un algoritmo de gradiente de política *on-policy* que, en cada iteración, actualiza la política maximizando un objetivo sustituto mediante ascenso por gradiente, limitando qué tanto puede cambiar la política en una sola actualización. Al igual que DDPG, PPO utiliza un esquema actor-crítico.

Una de las diferencias más importantes entre PPO y los algoritmos basados en Q-Learning es que **PPO NO UTILIZA REDES OBJETIVO**. En la implementación que se muestra en este cuadernillo, PPO-Clip, los problemas de estabilidad asociados a la tríada mortal se resuelven mediante una función de recorte del objetivo. La ecuación del objetivo es

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta)\, \hat{A}_t,\ \text{clip}\left(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon\right) \hat{A}_t \right) \right]$$

donde la razón de probabilidades es

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}$$

$$\theta_{k+1} = \arg\max_\theta \frac{1}{|\mathcal{D}_k| T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t),\quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

Donde:

- $\pi_\theta(a \mid s)$ es la política actual y $\pi_{\theta_{old}}(a \mid s)$ es la política del paso anterior, la que recolectó los datos.
- $\hat{A}_t$ es la **ventaja**, que se calcula como la diferencia entre el valor de la acción y el valor del estado: $A(s,a) = Q(s,a) - V(s)$.
- ϵ es el hiperparámetro de recorte (en el código, `eps_clip = 0.2`).

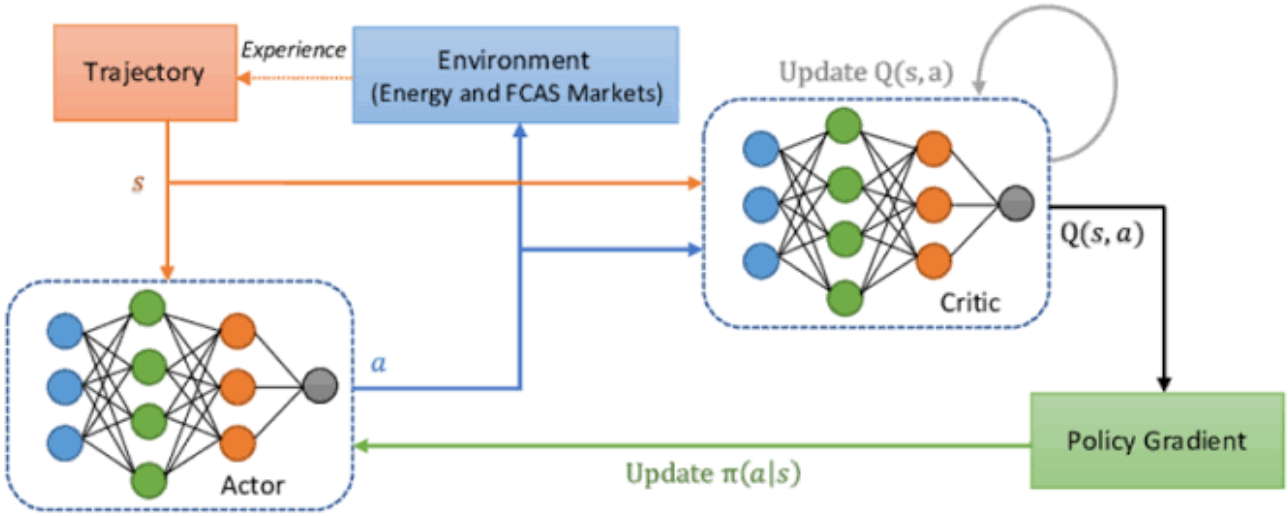
El término derecho de la ecuación puede escribirse también como $g(\epsilon, A)$, que se define por partes:

$$g(\epsilon, A) = \begin{cases} (1+\epsilon)\, A & \text{si } A \geq 0 \\ (1-\epsilon)\, A & \text{si } A < 0 \end{cases}$$

Con una función tan extraña, vale mucho la pena cuestionar: ¿por qué esto funciona?

Una analogía (generada por Gemini al preguntarle si podía explicarlo como si fuésemos niños de 5 años) es la siguiente. Imagine que está aprendiendo a andar en bicicleta. Si intentamos pedalear demasiado rápido y damos un giro gigantesco de golpe, podemos lastimarnos. El clip es como la mamá o papá que nos sostiene y no nos deja irnos por un precipicio. Volviendo al mundo del Machine Learning, el clipeo es como esa persona que nos detiene de dar pasos peligrosamente grandes.

Para continuar indagando en cómo funciona PPO, recordemos que la actualización se hace a través de una razón de probabilidades. Más formalmente, es una razón de políticas que nos da la probabilidad (o densidad de probabilidad) de escoger una acción determinada dado un estado. Cuando tomamos una buena acción con la política nueva, el numerador del cociente aumenta para esa acción, porque es más probable que el agente la escoja de nuevo. En cambio, si el agente hace algo malo, la probabilidad de escogerla en el futuro disminuye y el cociente se vuelve menor que 1. El problema surge cuando estos cambios son tan grandes que el algoritmo olvida lo que aprendió previamente. Es como si un niño en bicicleta le metiera el pie completo al pedal y se fuera por un barranco, sin alcanzar nunca su destino.



Una genialidad de PPO es que es un algoritmo altamente optimizado para implementarse con herramientas de Deep Learning, especialmente en Python. Las librerías TensorFlow y PyTorch están optimizadas para calcular gradientes mediante *backpropagation*, lo cual eficientiza considerablemente su evaluación.

En el pseudocódigo de PPO-Clip, después de modificar los parámetros de las redes del actor y del crítico, se utiliza un algoritmo de **ascenso por gradiente estocástico** para el actor y uno de **descenso por gradiente** para el crítico. En la práctica, estas fórmulas no se implementan línea por línea (a menos que queramos programar desde cero). Las librerías de Python para Deep Learning ya contienen **optimizadores** especializados para minimizar funciones de costo. Como estas herramientas están hechas para minimizar, el ascenso por gradiente se consigue invirtiendo el signo de la función objetivo: maximizar f es equivalente a minimizar $-f$. Por eso, en el código la pérdida de la política aparece con signo negativo.

> **Nota:** en `ppo.py`, ambas redes se optimizan con un único optimizador Adam sobre una pérdida combinada, en lugar de dos optimizadores separados como sugiere el pseudocódigo.

En esta implementación DDPG se realiza en un script separado `ddpg.py`

Algorithm 1 PPO-Clip

Input: initial policy parameters θ_0, initial value function parameters ϕ_0

for $k = 0, 1, 2, \ldots$ **do**

Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.

Compute rewards-to-go $\hat{R}_t$.

Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.

Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \;\; g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

end for

Referencia: Spinning Up – PPO

6. Entrenamiento

Finalmente llegamos al momento de la verdad: vamos a generar instancias de los algoritmos implementados en este cuadernillo y a entrenarlas en el entorno.

6.1. Aprendizaje por currículos

El **aprendizaje por currículos** (*curriculum learning*) es una técnica de aprendizaje automático en la que un agente aprende el comportamiento deseado en etapas de entrenamiento con una complejidad progresivamente mayor. En este caso, se utiliza esta técnica para que el robot aprenda a adaptarse a las incertidumbres de su entorno:

- Las masas, coordendas de centros de masa e inercias rotacionales pueden variar de una implementación a otra
- Los terrenos con texturas distintas tienen coeficientes de fricción distintos.
- El sensor de movimiento IMU puede producir lecturas ruidosas
- Los motores pueden responder con retardo o con ruido.
- Se pueden aplicar perturbaciones externas sobre el robot
- Las ruedas pueden tener crestas en su cubierta, y rodar sobre superficies irregulares
- Los motores pueden consumir un voltaje mayor o menor entre implementaciones.
- La ubicación del sensor de aceleración podría variar entre una implementación a otra.
- La posición inicial del robot puede variar

[Claude por favor revisa esa enumeración]

Para atender a cada una de estas implementaciones, se proponen las siguientes 7 etapas de entrenamiento que agrupan a estos factores.

Etapa 1: Balance Básico (Bucle Inicial)Objetivo: Enseñar al agente la función primaria de mantenerse en posición vertical.Configuración:No hay penalizaciones por desplazamiento horizontal ni por giros en el eje de guiñada (yaw).El agente se enfoca exclusivamente en no caerse, sin importar si se desplaza por el entorno.Se aplica una pequeña aleatorización en las condiciones iniciales (como una velocidad angular ligera al reiniciar el episodio) para aprender a recuperarse del desequilibrio.Etapa 2: Balance en el Origen (Estabilidad Estática)Objetivo: Lograr que el robot permanezca en un punto fijo mientras mantiene el equilibrio.Configuración:Se introducen coeficientes de penalización por movimiento lineal (X, Y) y por rotación (yaw).El agente aprende a mantenerse erguido reduciendo drásticamente cualquier desplazamiento o giro innecesario.Etapa 3: Introducción de Ruido en Sensórica y RetardosObjetivo: Iniciar la Randomización del Dominio en la lectura de los sensores.Configuración:Se inyecta ruido gaussiano a las lecturas del giroscopio, acelerómetro (ángulo de pitch y su velocidad) y a los encoders de las ruedas.Se introducen retardos aleatorios en la ejecución de las acciones mediante un buffer FIFO (simulando la latencia de la ejecución del firmware).Etapa 4: Empujes Aleatorios y Perturbaciones MecánicasObjetivo: Entrenar al robot para recuperarse de fuerzas externas e imperfecciones en los motores.Configuración:Se inyecta ruido en las señales de control enviadas a los actuadores.Se aplican empujes externos aleatorios (pushes) en las direcciones X e Y con una baja probabilidad por paso de tiempo, simulando baches, pequeñas colisiones o rozamientos.Etapa 5: Variación de Masa y FricciónObjetivo: Adaptar la red a variaciones de la superficie y del modelo 3D.Configuración:En cada episodio se varía aleatoriamente la masa y la densidad del chasis para compensar desequilibrios en el centro de masa real.Se modifica aleatoriamente el coeficiente de fricción del suelo y de las ruedas para tolerar desde alfombras hasta pisos de madera o hule.Etapa 6: Ganancia y Tolerancia de Voltaje de los MotoresObjetivo: Simular caídas de potencia y respuestas no lineales de la batería.Configuración:Se aleatoriza la ganancia (motor gain) del torque entre el 60% y el 100% de la capacidad nominal del motor.Esto prepara al agente para mantener el equilibrio incluso cuando la batería se desgasta o presenta caídas instantáneas de voltaje al encender los motores a alta velocidad.Etapa 7: Simulación de Imperfecciones Mecánicas (Tire Ridges)Objetivo: Modelar irregularidades físicas en la tracción de las llantas.Configuración:Se agregan pulsos aleatorios de torque (axle torques) con una probabilidad reducida en cada paso de tiempo.Simula el comportamiento de las estrías del neumático, vibraciones del chasis y juego mecánico en las cajas reductoras (backlash).

Finalmente también se desea que el robot sepa responder a comandos externos. Un comando se representa a través de dos entradas X y Y que juntas forman un vector de velocidad local. Si el comando es cero, el robot debería mantenerse en una misma posición, sin experimentar drift.

> **Pendiente:** completar la descripción de las etapas del currículo para este robot (por ejemplo, perturbaciones iniciales de ángulo o cambios en masa y fricción).